HYPERION AI

SYSTEM DESIGN DOCUMENT

# Async Python Pipelines at Scale

Reference architecture for 19+ interlocked Python pipelines with Celery, FastAPI, Claude, and dedicated DLQ

PREPARED FOR

Teams running many async Python tools in parallel

**Vitalijus Alsauskas**

Software and AI Engineer

Hyperion AI - Production AI engineering for founders and agencies

https://hyperionai.dev · 2026-04-18

# Async Python Pipelines at Scale — System Design

| | |
|---|---|
| **Prepared for** | Teams running 19+ async Python tools in parallel |
| **Prepared by** | Vitalijus Alsauskas · Hyperion AI |
| **Scope** | FastAPI gateway, Celery + Redis orchestration, isolated workers per tool, dedicated DLQ, structured logging with request IDs |

## 1. Problem Framing

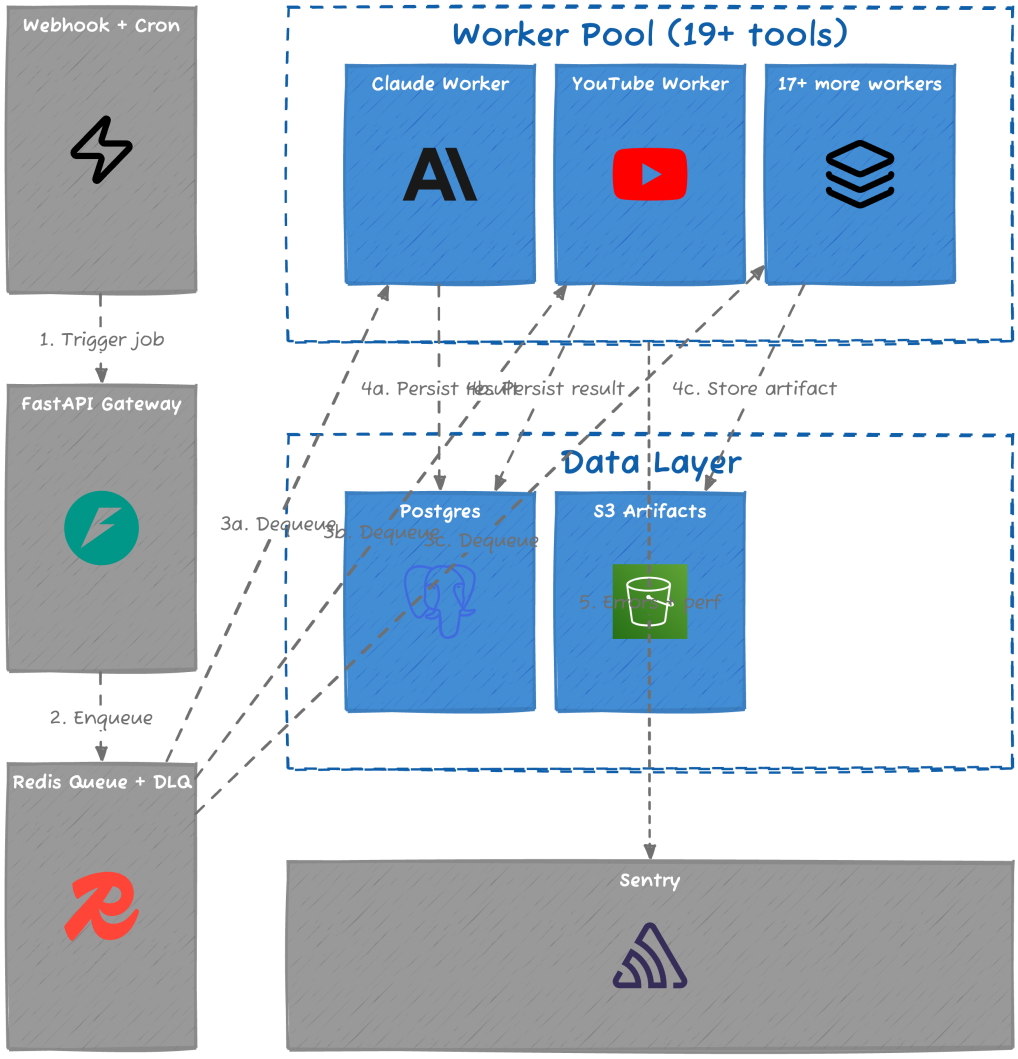
Async Python at 2-3 tools feels like a solved problem. At 10+ it falls over.

Common failure modes once a pipeline fleet grows:

- **One bad worker starves the rest** — the video-render queue backs up, takes down the LLM queue with it
- **Silent retry failures** — retries look like success, exhausted-retry jobs just vanish
- **Scattered logs across 19 streams** — 3am debugging turns into grep across a dozen services
- **No one knows who called what** — no trace of which trigger spawned which worker + external API calls

This design shows the pattern that **holds at scale**: FastAPI front-door, Celery workers isolated per tool, a dedicated DLQ that never silently loses jobs, and `request_id` propagation that makes one grep enough to reconstruct any job's path.

## 2. System Diagram



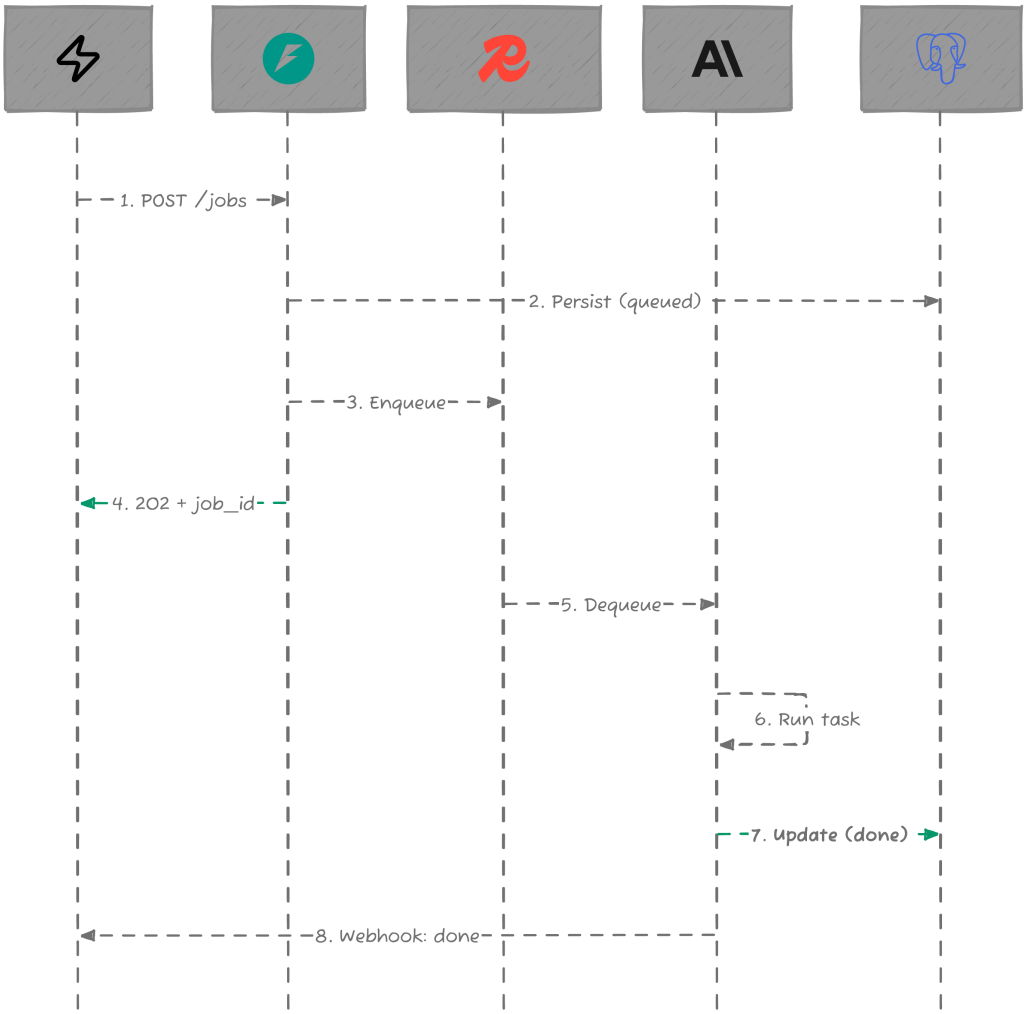

Every pipeline tool runs as **its own isolated Celery worker**. A video-render failure never blocks Claude extraction. Every exhausted-retry job lands in a **dedicated DLQ** — never silently lost.

Source: `docs/architecture.d2`

## 3. Tool Choices

| LAYER | PICK | WHY THIS | ALTERNATIVES CONSIDERED |
|---|---|---|---|
| **API Gateway** | FastAPI + Pydantic | Async-native, OpenAPI auto, strong typing from day one | Flask (no async), Django (overkill), aiohttp (weaker DX) |
| **Task Queue** | Celery + Redis | Mature, battle-tested, fine-grained retry + DLQ control | RQ (simpler, weaker scheduling), Dramatiq (smaller ecosystem), Arq (young) |
| **Broker** | Redis | Fast, cheap, doubles as cache | RabbitMQ (more ops), SQS (cloud lock-in) |
| **Scheduler** | Celery Beat | Co-located with Celery | APScheduler (extra service), cron (no retry visibility) |
| **Workers** | Celery, one class per tool | Isolation, per-tool concurrency tuning | Monolithic worker (one bad job starves rest) |
| **Observability** | Structlog + Sentry + Flower | JSON logs, exception context, queue visibility | ELK (overkill), DataDog (expensive) |
| **DB** | Postgres | Job state + results, strong consistency | Mongo (worse for relational graphs), Redis-only (no durability) |
| **Object Storage** | S3 | Standard, cheap | Local disk (doesn't scale) |
| **LLM** | Claude 3.5 Sonnet | Strong structured output, zero-retention | GPT-4o (close, retention less favorable), local Llama (unreliable at scale) |

## 4. Data Flow




### Happy path

1. **Trigger** fires (cron, webhook, or manual CLI)
2. **API** validates and persists job row with `status=queued`
3. Job ID **enqueued** to Redis broker
4. **Worker** for the appropriate tool dequeues
5. Worker calls external APIs (Claude, YouTube, ffmpeg, etc.)
6. Artifacts land in **S3**, small results in **Postgres**
7. Job row updated to `status=done`
8. Outbound **webhook** fires to the downstream system

### Error paths

| FAILURE TYPE | HANDLING |
| --- | --- |
| **Retryable** (rate limit, network timeout, 5xx) | Caught, logged to Sentry with context, requeued with exponential backoff up to retry budget (default 3) |
| **Non-retryable** (malformed input, auth failure) | Marked failed immediately, logged, no retry |
| **Exhausted retries** | Moved to **DLQ**, `status=failed` with error context, Sentry critical alert, failure webhook fires |
| **Worker crash mid-job** | Job ack only after successful commit — crashed worker means job returns to queue automatically |
| **DB write failure** | Transaction rollback, `status=failed-transient`, retried |

## 5. Component Breakdown

| COMPONENT | STACK | RESPONSIBILITY |
| --- | --- | --- |
| **API Gateway** | FastAPI + Pydantic + Structlog | Public job submission + status, auth middleware, audit log |
| **Celery Beat** | Celery scheduler | Cron-triggered job submission |
| **Worker Pool** | One Python module per tool under `workers/` | Each worker declares concurrency + retry + timeout in code |
| **DLQ Monitor** | Lightweight service | Reads DLQ, fires Sentry alerts, exposes `replay` endpoint |
| **Observability Stack** | Structlog + Sentry SDK + Flower | JSON logs, async-aware exception capture, queue visibility |
| **Data Layer** | SQLAlchemy + Alembic + boto3 + redis-py | Postgres schema, S3 access, broker + cache |

## 6. Implementation Phases

| PHASE | WEEKS | SHIPS | KEY MILESTONES |
|---|---|---|---|
| **1. Foundation** | 1-2 | Skeleton with one pipeline tool end-to-end | `/jobs` endpoint, Celery + Redis + Beat via docker-compose, one worker class, Structlog + Sentry + Flower, DLQ monitor, integration tests |
| **2. Pipeline Coverage** | 3-4 | All 19+ tools migrated into isolated Celery workers | Per-tool retry/timeout policies, load test at 3× peak, per-worker runbook stubs |
| **3. Production Polish** | 5-6 | Observability refined, DLQ replay flow, prod deploy | Request ID propagation across all workers, DLQ replay CLI, full runbook, `/replay-dlq` Claude Code command, GitHub Actions deploy |

## 7. Risks & Mitigations

| # | RISK | MITIGATION |
|---|---|---|
| 1 | **Malformed input blocks the whole queue** | Worker isolation — each tool has its own queue + pool. Hard timeouts on every task prevent hung APIs from pinning workers. |
| 2 | **Retries mask silent failures** — jobs vanish | Dedicated DLQ. Every exhausted-retry job lands there, fires Sentry critical, marks `status=failed` with full context, fires failure webhook. Never silently lost. |
| 3 | **3am debugging across 19 log streams** | Structlog + `request_id` attached at ingress, propagated everywhere. One `grep request_id=abc123` surfaces the full job history. |
| 4 | **Claude API rate limits during peak** | Per-worker concurrency caps + exponential backoff on 429 + Claude-specific rate limiter. Haiku fallback under extreme load. |
| 5 | **Autoscaling misconfigured** — jobs pile up at night | Queue-depth alerts (warn 100, critical 500). Autoscaling tied to queue depth not CPU. Flower accessible from phone. |

## 8. What I Need From You

To move into Phase 1, a few inputs from your side:

- **Read access to the existing codebase** (or a representative sample) — so I can read before proposing changes
- **List of the 19+ tools** — one-liner per tool on what each does
- **Current deployment target** — Hetzner VM? Container Apps? k8s? bare metal?
- **Ranked pain points** — is 3am debugging the worst, or rate limits, or onboarding new pipelines?

- **30-minute call** — to walk through this doc against your actual setup and tighten scope before code ships

---

*This document is yours to keep regardless of whether we end up working together. The architecture and tool choices here should save any developer who inherits this work a week of deliberation.*